bit of calculation in a form. This time period now gets referred to as Web 1.0. As computers got faster and browsers added features, new degrees of interactivity got added to web pages. Many tools grew out of this time period, and web pages evolved into fully contained applications. Meanwhile, Django kept chugging along in its rather old-school way.

Web 2.0 can require a lot of JavaScript tooling. Frameworks like React and Angular mean potentially writing in TypeScript and adding a compilation stage to your web development. Libraries like Vue.js straddle the line, allowing you to mix and match 1.0-style pages with 2.0-style dynamics. If you want to build a web-based spreadsheet, these heavy-duty frameworks are a must. You wouldn't want the page to reload every time the user edited a cell; a high degree of interactivity is required. Single-page applications are really full programs running in your browser, rather than being paged based. The vast majority of business applications don't need this degree of interactivity. Most business applications really are just CRUD interfaces to a database. A sprinkling of interactivity is often enough.

HTMX is a JavaScript library that can meet a significant percentage of your dynamic web page needs. It works by adding the capability of replacing a portion of a web page through a URL call. Essentially, it brings the web 1.0 principle of visiting a URL and getting a page back to parts of the page. This works quite well with Django. HTMX modifies a loaded page on the fly by fetching part of a page from a Django view.

HTMX isn't meant to replace the heavier frameworks for situations like building a web-based spreadsheet, but it does provide a degree of interaction with very little JavaScript. The structure of your project can be very similar to Web 1.0, while the dynamic interactions on your pages get brought into the 2.0 era.

12.2 HTMX attributes

The HTMX library adds a set of attributes to HTML that you can place on almost any tag. These new attributes center around HTTP methods, and so are very similar to the REST concepts covered in chapter 11. When a user interacts with an HTML tag containing an HTMX attribute, the corresponding HTTP method gets invoked. Other HTMX attributes specify what to do with the response from the REST-like call. A typical example is to replace the activated tag with the response from the server. When using HTMX with Django, the only difference is that some Django views now return part of a web page, rather than a complete one. These component pieces are known as *partials*.

Consider a web page that shows the user the details of a random song and a link that swaps the song for another one when clicked. In the Web 1.0 world, clicking a link loads the whole page again to fetch a single new song. With HTMX, the link executes a GET to fetch an HTML partial containing the new content, which HTMX then replaces in the page for you. Dynamic!

All HTMX attributes begin with an `hx-` prefix. The `hx-get` performs an HTTP GET operation when the attached tag gets interacted with. The `hx-target` and